Delta Tables in Databricks — A Practical Guide for Credit Risk Model Development

Audience: Senior Credit Risk Model Developer, new to creating and managing Delta tables. **Goal:** Extract data (existing Delta tables, CSV, Excel), transform and aggregate it, and store results in your own Delta tables — including appending, updating, access control, and version history. **Time budget:** Approximately 2 hours total, including setup and reading this guide. **Level:** Fundamentals. No streaming, no Lakeflow pipelines, no performance tuning.

Part 1 — Setting Up Databricks Free Edition (about 20 minutes)

1.1 What is Databricks Free Edition?

Databricks Free Edition is a permanently free, personal Databricks workspace for learning and experimentation. It replaced the legacy Community Edition, which was retired in 2025. It runs on serverless compute only (no clusters to configure), has Unity Catalog enabled by default, and includes default storage. It is licensed for non-commercial use only, which is exactly right for training on synthetic data. Do not upload real customer data.

1.2 What you need to provide

Item	Requirement
Email address	Any personal or work email; verified via one-time password (OTP). Alternatively sign in with Google or Microsoft.
Payment details	None. No credit card, no cloud account, no time limit.
Software	A browser only. Nothing is installed locally.
Data	Small synthetic files, which Part 3 of this guide provides.

1.3 Sign-up steps

1. Go to the Databricks Free Edition sign-up page (linked from databricks.com/learn/free-edition; the docs page "Sign up for Databricks Free Edition" contains the direct link).
2. Choose a sign-up method: email OTP, Google, or Microsoft. Avoid your corporate SSO because Free Edition does not support SSO; your work Databricks account is a separate account anyway.
3. Verify your email if prompted.
4. Your workspace opens after a few minutes. It contains serverless compute and default storage immediately — you never start a cluster.
5. Optional but recommended: click "Verify identity" in the top bar and complete LinkedIn verification. This raises some usage limits. It is not required for this guide.

1.4 What is included, and what is limited

Included (relevant to your goals): notebooks with PySpark and Spark SQL, the SQL editor, one small SQL warehouse, Unity Catalog with Catalog Explorer, schemas and volumes, Delta tables with full version history (time travel), MERGE, GRANT syntax, and file upload through the UI.

Key limitations:

Limitation	Consequence for this guide
Serverless compute only, limited size and daily usage quota	Keep datasets small (this guide uses under 40 rows). If you exceed the quota, compute pauses until the next day; your data is not deleted.
Outbound internet access is restricted	Do not try to download files from the web inside a notebook. Upload files through the UI instead (Part 3, Step 2).
One workspace, one catalog, single user	You practice GRANT syntax, but you cannot test real sharing with colleagues. This is the one topic to complete at work.
R and Scala are not supported	This guide uses Python and SQL only, which matches your use case.
Non-commercial use only	Fine for training. Never put real bank data here.
Accounts inactive for a prolonged period may be deleted	Finish the guide in one session; download your notebook afterwards if you want to keep it.

1.5 Does Free Edition help you learn the features you need?

Yes, for almost everything. Creating schemas, creating and appending to Delta tables, MERGE updates, time travel, file uploads, transformations, aggregations, and grant syntax can all be practiced fully and identically to your work environment (assuming your work workspace also uses Unity Catalog). Only two aspects are limited: real multi-user access control (single-user account) and scheduled/automated workloads. You can transfer those skills at work directly, where you now have schema rights.

Part 2 — The Feature Map for Your Use Case (about 10 minutes of reading)

The features below are ordered along your actual workflow: land files, store them as tables, transform, update, control versions, control access, consume.

#	Feature	Where in the process	What it does	Why it is useful for you
1	Workspace and serverless compute	Always	The web environment you work in; serverless compute starts automatically per notebook cell.	No cluster administration; you focus on data and code.
2	Unity Catalog and Catalog Explorer	Start	The governance layer that organizes everything in a three-level namespace: catalog.schema.table. It also manages permissions.	At work this is how your schemas are structured. Knowing where tables live is the first skill.
3	Schemas	Start	A schema is a folder-like namespace inside a catalog that groups related tables and volumes.	Your new permission is schema creation. A clean schema separates raw, transformed, and result tables.
4	Volumes	Data landing	A Unity Catalog-managed storage location for files (CSV, Excel) at /Volumes/catalog/schema/volume/.	The standard way to upload local files. From there, code reads them.
5	Notebooks (PySpark and Spark SQL)	Everywhere	The interactive work surface; cells run Python or SQL against serverless compute.	Your workbench for all extraction, transformation, and loading.
6	Reading files: spark.read, read_files, pandas	Extract	Loads CSV into a Spark DataFrame; Excel via pandas with openpyxl (or built-in Excel support).	Your entry point for uploaded Excel and CSV files.
7		Store	Persists a DataFrame or query result as a managed Delta table inside your schema.	

#	Feature	Where in the process	What it does	Why it is useful for you
	Creating Delta tables: saveAsTable, CREATE TABLE, CTAS			Delta is the default format; every table you create is Delta with ACID guarantees.
8	Transform and aggregate: DataFrame API, Spark SQL, CTAS	Transform	Filter, join, group, aggregate; store results with CREATE TABLE ... AS SELECT.	Risk data for rating calibrations is exactly this: rates by segment, exposure sums, vintage tables.
9	Appending data: append mode, INSERT INTO	Update	Adds new rows to an existing Delta table without touching old rows.	Adding a new observation year or a new data delivery to your table.
10	MERGE INTO (upsert)	Update	Matches rows by key and updates them, inserts non-matching rows.	Corrections and restated data (a cured default, a restated rating) without rebuilding the table.
11	Time travel: DESCRIBE HISTORY, VERSION AS OF, RESTORE	Version control	Every change to a Delta table creates a version. You can inspect history, query old versions, and restore.	Auditability and undo — you can prove what the table looked like when a model was calibrated.
12	Table comments and constraints	Quality	Column comments, table comment, NOT NULL and CHECK constraints.	Documentation and basic data quality gates that a validator will appreciate.
13	GRANT / REVOKE, ownership	Access control	Unity Catalog privileges (SELECT, MODIFY, USE SCHEMA) control who reads or writes a table.	Making results available to colleagues read-only while you keep write access.
14	SQL editor and warehouse	Consume	Ad-hoc SQL against your tables with a small SQL warehouse; quick visualizations.	Fast checks and simple charts without a notebook.

Part 3 — Step-by-Step Training: Pet Project "Mini Loan Book"

You build a complete, self-contained mini risk-data pipeline: you land two small files, build a Delta table of loan-year observations, append a new year, apply corrections with MERGE, aggregate default rates by rating class, and finish with time travel and grants. Everything runs in your Free Edition workspace with the data below.

Project data

These files are provided as CSV downloads with the names `01_loans_2025.csv`, `02_loans_2026.csv` (plus an Excel copy `02_loans_2026.xlsx`), and `03_corrections.csv`. If you downloaded them, upload them under exactly these names; all code in this guide uses these names.

`01_loans_2025.csv` — 12 loan-year observations:

```
loan_id,obs_year,segment,region,product,loan_amount,rating,status
L001,2025,Retail,Bavaria,Mortgage,250000,2,active
L002,2025,Retail,Berlin,Consumer,15000,5,active
L003,2025,SME,Hesse,Business,120000,3,active
L004,2025,Retail,Bavaria,Consumer,8000,7,default
L005,2025,SME,Berlin,Business,200000,6,active
L006,2025,Corporate,Hesse,Business,500000,1,active
```

```
L007,2025,Retail,Berlin,Mortgage,180000,4,active
L008,2025,SME,Bavaria,Business,90000,8,default
L009,2025,Retail,Hesse,Consumer,12000,6,active
L010,2025,Corporate,Berlin,Business,350000,2,active
L011,2025,Retail,Bavaria,Consumer,20000,9,default
L012,2025,SME,Hesse,Business,150000,5,active
```

`02_loans_2026.csv` — a new delivery for the next observation year (also save as `02_loans_2026.xlsx` for the Excel step in Step 4):

```
loan_id,obs_year,segment,region,product,loan_amount,rating,status
L013,2026,Retail,Bavaria,Consumer,10000,4,active
L014,2026,SME,Berlin,Business,175000,3,active
L015,2026,Retail,Hesse,Mortgage,220000,5,active
L016,2026,Corporate,Bavaria,Business,400000,2,active
L017,2026,Retail,Berlin,Consumer,9000,10,default
L018,2026,SME,Hesse,Business,130000,7,active
```

`03_corrections.csv` — status corrections for the MERGE step:

```
loan_id,obs_year,new_status
L004,2025,cured
L009,2025,default
L015,2026,default
```

Time plan: Steps 0–2 about 20 minutes, Step 3 about 15, Step 4 about 15, Step 5 about 20, Step 6 about 10, Step 7 about 10, Step 8 about 10, Step 9 about 5.

Step 0 — Orientation (5 minutes)

After sign-up, look at the left sidebar. Workspace holds notebooks, Catalog opens Catalog Explorer, SQL opens the SQL editor, Compute shows the serverless resources. Note your catalog name: click Catalog in the sidebar. Under "My organization" you see your workspace catalog, which in Free Edition is literally named `workspace`, and the read-only `system` catalog. Expand `workspace` to find the `default` schema. Ignore the `samples` catalog — it only contains Databricks demo data. In all code below, substitute `<catalog>` with `workspace`.

Step 1 — Create your schema and volume (5 minutes)

Create a notebook: Workspace, then Create, Notebook, default language Python. Attach it to serverless compute if asked. Run this first cell (Shift+Enter runs a cell):

```python
catalog = "workspace" # your workspace catalog in Free Edition (see Step 0)
schema = "risk_training"
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.{schema}")
spark.sql(f"CREATE VOLUME IF NOT EXISTS {catalog}.{schema}.raw_files")
print(f"Volume path: /Volumes/{catalog}/{schema}/raw_files")
```

Checkpoint: open Catalog, expand your catalog; you see the schema `risk_training` with the volume `raw_files`. The three-level namespace is already the core mental model: everything is catalog.schema.object.

Step 2 — Upload the three files (10 minutes)

In Catalog Explorer, navigate to the volume `risk_training.raw_files` and click Upload (or Add data, then Upload files). Upload all three files. Volume paths have the form:

```
/Volumes/<catalog>/risk_training/raw_files/01_loans_2025.csv
```

Note: because Free Edition restricts outbound internet access, uploading through this UI is the only reliable way to bring local files in.

Step 3 — First Delta table from CSV (15 minutes)

Read the CSV into a Spark DataFrame, inspect it, and write it as a managed Delta table.

```
catalog = "workspace" # same value as in Step 1
schema = "risk_training"
df = (spark.read
 .option("header", True)
 .option("inferSchema", True)
 .csv(f"/Volumes/{catalog}/{schema}/raw_files/01_loans_2025.csv"))
df.printSchema() # check inferred types
df.show(5)
df.write.mode("overwrite").saveAsTable(f"{catalog}.{schema}.loans_raw")
```

Inspect the result in Catalog Explorer: table `loans_raw`, tabs Overview (schema), Sample Data, and History (already one version). The SQL equivalent of the last line, useful at work:

```
CREATE OR REPLACE TABLE <catalog>.risk_training.loans_raw AS
SELECT * FROM read_files(
 '/Volumes/<catalog>/risk_training/raw_files/01_loans_2025.csv',
 format => 'csv', header => true, inferSchema => true);
```

Note the write mode: `overwrite` replaces the table (idempotent re-runs during development); `append` adds rows (Step 4); `errorIfExists` is the safe default for production tables.

Step 4 — Excel file and appending a new year (15 minutes)

The Excel copy `02_loans_2026.xlsx` is already in the volume (uploaded in Step 2); skip the conversion if you uploaded it then. Read the Excel file with pandas (installing openpyxl works on serverless) and append it to the Delta table. The engine matters: pandas uses `openpyxl` for modern `.xlsx` files and `xlrd` for legacy `.xls` files — the two are not interchangeable.

```
%pip install openpyxl
```

```
import pandas as pd
pdf = pd.read_excel(f"/Volumes/{catalog}/{schema}/raw_files/02_loans_2026.xlsx",
 engine="openpyxl")
df_new = spark.createDataFrame(pdf)
df_new.write.mode("append").saveAsTable(f"{catalog}.{schema}.loans_raw")
spark.table(f"{catalog}.{schema}.loans_raw").count() # expect 18
```

Checkpoint: the count is 18; Catalog Explorer, History tab of `loans_raw` shows two versions. This append pattern is exactly what you will do at work when a new monthly or yearly data delivery arrives.

Alternative for computers without Microsoft Excel: LibreOffice Calc can save spreadsheets as `.xlsx` (file type "Excel 2007-365"), so the openpyxl path above works unchanged. If the file exists only as legacy `.xls`, install and use `xlrd` instead:

```
%pip install xlrd
```

```
import pandas as pd
pdf = pd.read_excel(f"/Volumes/{catalog}/{schema}/raw_files/02_loans_2026.xls",
 engine="xlrd")
df_new = spark.createDataFrame(pdf)
df_new.write.mode("append").saveAsTable(f"{catalog}.{schema}.loans_raw")
spark.table(f"{catalog}.{schema}.loans_raw").count() # expect 18
```

Step 5 — Transform and aggregate: the results table (20 minutes)

Compute default rates by rating class and by segment, and store them in a new Delta table. This is the analyst heart of the workflow: SQL you already know, persisted as a governed table.

```
result = spark.sql(f"""

SELECT rating,

count(*) AS n_loans,

sum(loan_amount) AS exposure,

avg(CASE WHEN status = 'default' THEN 1.0 ELSE 0.0 END) AS default_rate

FROM {catalog}.{schema}.loans_raw

GROUP BY rating

ORDER BY rating

""")
result.show()
result.write.mode("overwrite").saveAsTable(f"{catalog}.{schema}.default_rates_by_rating")
```

Add a table comment so future users (including validators) understand the content:

```
COMMENT ON TABLE <catalog>.risk_training.default_rates_by_rating IS
'Default rate and exposure by rating class, aggregated from loans_raw. Training
data, synthetic.';
```

Also run one segment-level aggregation with a filter and write it as `default_rates_by_segment` (same pattern, WHERE segment = ... or GROUP BY segment). Reading data from an existing Delta table is just `spark.table("catalog.schema.table")` or SELECT in SQL — the same pattern you use at work for the source tables you are allowed to read.

Step 6 — Corrections with MERGE (10 minutes)

Load the corrections file and update matching rows. MERGE is the Delta feature that makes updates safe and auditable: rows are matched by key, matched rows are updated, and everything is one atomic transaction that creates a new table version.

```
corr = (spark.read
 .option("header", True)
 .option("inferSchema", True)
 .csv(f"/Volumes/{catalog}/{schema}/raw_files/03_corrections.csv"))
corr.createOrReplaceTempView("corrections")
spark.sql(f"""

MERGE INTO {catalog}.{schema}.loans_raw t

USING corrections c

ON t.loan_id = c.loan_id AND t.obs_year = c.obs_year

WHEN MATCHED THEN UPDATE SET t.status = c.new_status

""")
```

Checkpoint: `SELECT * FROM ... WHERE loan_id IN ('L004','L009','L015')` shows the three corrected statuses (cured, default, default). Note the merge key: for loan-year data you need the composite key loan_id plus obs_year.

Step 7 — Version control with time travel (10 minutes)

Every write, append, and merge created a version. Inspect the history, query an old version, and restore if needed.

```
spark.sql(f"DESCRIBE HISTORY {catalog}.{schema}.loans_raw").show(truncate=False)
```

```
-- what did the table look like before the merge?
SELECT * FROM <catalog>.risk_training.loans_raw VERSION AS OF 2 WHERE
loan_id = 'L004';
```

```
-- undo the merge entirely (creates yet another version)
RESTORE TABLE <catalog>.risk_training.loans_raw TO VERSION AS OF 2;
```

Checkpoint: after RESTORE, loan L004 is back to status `default`. For a model developer, time travel means reproducibility: you can document exactly which table version a calibration used.

Step 8 — Access management with GRANT (10 minutes)

Unity Catalog privileges control access. In Free Edition you are the only user, so this is syntax practice; at work, replace the principal with real users or groups.

```
GRANT SELECT ON TABLE <catalog>.risk_training.default_rates_by_rating TO `user@example.com`;
SHOW GRANTS ON TABLE <catalog>.risk_training.default_rates_by_rating;
REVOKE SELECT ON TABLE <catalog>.risk_training.default_rates_by_rating FROM `user@example.com`;
```

Concepts to remember: USE CATALOG and USE SCHEMA let a user see that objects exist; SELECT allows reading; MODIFY allows writing; the table owner can grant. At work, ask your administrator which group your consumers belong to, then grant SELECT to that group only — results tables are typically read-only for everyone except the owner.

Step 9 — Consume in the SQL editor and wrap up (5 minutes)

Open SQL, New query, and run a final query against your results table:

```
SELECT * FROM <catalog>.risk_training.default_rates_by_rating ORDER BY rating;
```

Use the plus icon under the result grid to add a quick visualization (bar chart of default_rate by rating). This closes the loop: land files, store as Delta, transform, update, version, govern, consume.

What you have built

Object	Type	Content
risk_training.raw_files	Volume	Three uploaded source files
risk_training.loans_raw	Delta table	18 loan-year observations, merged corrections, 3+ versions
risk_training.default_rates_by_rating	Delta table	Aggregated default rates by rating class
risk_training.default_rates_by_segment	Delta table	Segment-level aggregation

---

Appendix — Quick Reference

Task	PySpark	SQL
Create schema	spark.sql("CREATE SCHEMA ...")	CREATE SCHEMA catalog.schema
Read CSV	spark.read.option(...).csv(path)	read_files(path, format ⇒ 'csv', ...)
Read Excel	pd.read_excel(path) then spark.createDataFrame	(built-in Excel support on recent runtimes)
Create table	df.write.saveAsTable(name)	CREATE TABLE name AS SELECT
Append	df.write.mode("append").saveAsTable(name)	INSERT INTO name SELECT

Task	PySpark	SQL
Update by key	(use SQL)	MERGE INTO … WHEN MATCHED THEN UPDATE
Delete rows	(use SQL)	DELETE FROM name WHERE …
History	spark.sql("DESCRIBE HISTORY t")	DESCRIBE HISTORY t
Old version	spark.read.option("versionAsOf", 2).table(name)	SELECT … VERSION AS OF 2
Restore	(use SQL)	RESTORE TABLE t TO VERSION AS OF 2
Grant	(use SQL)	GRANT SELECT ON TABLE t TO principal

Sources: Databricks documentation on Free Edition sign-up, Free Edition limitations, working with files in Unity Catalog volumes, reading Excel files, and the tutorial "Import and visualize CSV data from a notebook".

Appendix B — Reproducing the Pipeline with Genie Code

Genie Code is the Databricks in-product assistant that writes and explains notebook code. You can use it to generate the entire pet project pipeline from a single prompt — a realistic preview of how you will work with AI assistance at work. Open a fresh Python notebook, open Genie Code, paste the prompt below, and review every generated cell before running it: read the code, then execute. The suffix `_by_Gini` isolates everything Genie creates from your manually built tables, so you can compare the two approaches side by side.

Assumption for this prompt: the three source files already sit in the volume, exactly as in your run: `01_loans_2025.csv`, `02_loans_2026.xls` (legacy Excel), and `03_corrections.csv` under `/Volumes/workspace/risk_training/raw_files`. If your Excel file is `.xlsx` instead, change the file name and engine in the prompt accordingly (openpyxl instead of xlrd).

Prompt to paste into Genie Code:

```
Context: I am working in a Databricks Free Edition workspace with Unity Catalog.
Catalog: workspace. Schema: risk_training. Files are in the volume
/Volumes/workspace/risk_training/raw_files with these contents:
- 01_loans_2025.csv: header row, columns
 loan_id,obs_year,segment,region,product,loan_amount,rating,status (12 rows).
- 02_loans_2026.xls: legacy Excel file, same columns (6 rows). Read it with
 pandas using engine="xlrd"; install xlrd with %pip install first.
- 03_corrections.csv: header row, columns loan_id,obs_year,new_status (3 rows).
Task: build a small risk-data pipeline. Every table you create or modify must
carry the suffix _by_Gini so my existing tables remain untouched. Do not run any
GRANT, REVOKE, or other access-control statements. Use PySpark and Spark SQL,
use the full three-part namespace workspace.risk_training.<table> everywhere,
and print the row count after each write.
Steps:
1. Read 01_loans_2025.csv with header and schema inference, print the schema
 and the first rows, and write it as the Delta table
 workspace.risk_training.loans_raw_by_Gini (mode overwrite).
2. Read 02_loans_2026.xls with pandas (engine xlrd), convert it to a Spark
 DataFrame, and append it to workspace.risk_training.loans_raw_by_Gini
 (mode append). Show the total row count and confirm it is 18.
3. From loans_raw_by_Gini, aggregate per rating class: number of loans,
 total exposure (sum of loan_amount), and the default rate
```

```
 (share of rows with status = 'default'). Store the result as
 workspace.risk_training.default_rates_by_rating_by_Gini (mode overwrite)
 and display it sorted by rating.
4. From loans_raw_by_Gini, aggregate the same three metrics by segment and
 store the result as
 workspace.risk_training.default_rates_by_segment_by_Gini (mode overwrite).
 Display the result.
5. Add a table comment to both result tables:
 'Training data, synthetic. Aggregated from loans_raw_by_Gini.'
6. Read 03_corrections.csv and MERGE it INTO
 workspace.risk_training.loans_raw_by_Gini, matching on loan_id and
 obs_year, updating status with new_status for matched rows. Afterwards
 display the rows for loan_ids L004, L009, and L015 and confirm the
 statuses are cured, default, and default.
7. Show the version history of loans_raw_by_Gini (DESCRIBE HISTORY) and
 explain in one sentence per version what change it corresponds to.
8. Query the version of loans_raw_by_Gini from before the merge to show the
 original status of loan L004. Then RESTORE the table to that pre-merge
 version and confirm that L004 has its pre-correction status again.
Rules: never modify or drop any table without the suffix _by_Gini. If a step
fails, show the error message and provide corrected code. Explain each cell
briefly so I can follow the logic.
```

What to compare after the run: check Genie's `loans_raw_by_Gini` history (expect roughly the same sequence of versions as yours: create, append, merge, restore), and compare the two `default_rates` tables for identical numbers. Differences are usually caused by reading the Excel file with the wrong engine or by aggregation order — both are instructive to debug with Genie.

Suggested next steps after this guide (optional)

1. Repeat the guide next week from memory; muscle memory matters more than breadth.
2. At work, request a personal schema and rebuild the same pipeline there, reading one real source table you are allowed to access.
3. When comfortable, look into CREATE OR REFRESH STREAMING TABLE for automated ingestion, and Lakeflow jobs for scheduling — only after the manual workflow feels routine.